

Visión por Computador Aplicada a Software

—Módulo 4 · Visión por Computador aplicada a proyectos de programación

Visión por Computador — Fundamentos e Investigación

Nombre del alumno/a	Angel Echenique
Fecha	26/04/2026
☐ Enlace GitHub del repositorio	

☐ Instrucciones: Responde con lo que hemos visto en clase. Puedes consultar tus apuntes. No hay que escribir código, ¡solo demostrar que entendiste! Algunas preguntas requieren investigación adicional.

BLOQUE 1 — ¿Qué permite la Visión por Computador en una aplicación? (3.1)

En clase vimos las capacidades principales que la CV aporta a las aplicaciones. Completa la tabla:

Nº	Capacidad CV	¿Qué hace exactamente?	Ejemplo concreto de uso
1	Clasificar imágenes	Asigna una categoría a una imagen según su contenido visual.	Una app que etiqueta fotos como "producto", "documento" o "persona".
2	Detectar objetos	Identifica y localiza elementos concretos dentro de una imagen.	Una app que cuenta coches en una calle o detecta herramientas en una mesa.
3	Extraer texto (OCR)	Convierte el texto que aparece en una imagen en texto digital editable.	Leer un ticket, una factura, una matrícula o un cartel.
4	Describir contenido	Genera una descripción en lenguaje natural de lo que aparece en la imagen.	Una app que describe una escena para una persona con discapacidad visual.
5	Moderar contenido inapropiado	Detecta imágenes con violencia, desnudos o contenido sensible para filtrarlas o bloquearlas.	Moderación automática en una red social o plataforma educativa.

6	Multimodal (texto + imagen)	Permite analizar una imagen junto con una instrucción o pregunta en texto para responder con contexto.	Subir la foto de un cartel y preguntar: "¿qué dice y qué significa?"
---	-----------------------------	--	--

1.2 — Las capas del análisis de imagen

La visión por computador va más allá de 'ver una foto'. Explica con tus propias palabras cada capa de análisis:

Tipo de análisis	¿Qué analiza exactamente?	Ejemplo práctico
Análisis de contenido	Analiza qué elementos aparecen en la imagen, como objetos, personas o animales.	Detectar que en una foto hay un coche, dos personas y una bicicleta.
Análisis de contexto	Analiza la situación o el entorno en el que aparecen esos elementos.	Reconocer que una persona está en una oficina, en la calle o en una cocina.
Análisis de calidad	Evalúa si la imagen tiene suficiente nitidez, luz o resolución para procesarla correctamente.	Comprobar si una foto de un DNI está borrosa antes de aceptarla.
Análisis de texto incrustado	Detecta y extrae el texto que aparece dentro de la imagen.	

1.3 — Completa la frase clave

"La visión por computador permite que las aplicaciones interpreten imágenes igual que el NLP interpreta textos.

Gracias a las APIs modernas, cualquier software puede analizar imágenes sin entrenar modelos propios."

BLOQUE 2 — Técnicas CV principales y cuándo usarlas (3.2 + 3.7)

Une cada técnica de visión por computador con su descripción, la API recomendada y cuándo usarla:

Técnica CV	¿Qué hace?	API recomendada	¿Cuándo usarla?
OCR	Extrae texto que aparece dentro de una imagen y lo convierte en texto digital.	Gemini (multimodal) o Azure Vision.	Cuando necesitas leer tickets, facturas, carteles, matrículas o documentos escaneados.
Clasificación de imágenes	Asigna una categoría a una imagen según su contenido visual.	Hugging Face ViT o OpenAI en clasificación zero-shot.	Cuando necesitas etiquetar imágenes como "producto", "documento", "persona" o categorías similares.
Detección de objetos	Identifica objetos concretos y además indica dónde están dentro de la imagen.	YOLO, Hugging Face o Azure Vision.	Cuando hace falta localizar y contar elementos, por ejemplo coches, herramientas o productos.
Descripción de imágenes	Genera una explicación en lenguaje natural de lo que aparece en la imagen.	GPT-4o o Claude 3.	Cuando la app debe explicar una escena, resumir una foto o ayudar en accesibilidad.
Moderación visual	Detecta contenido violento, adulto o sensible para filtrarlo o bloquearlo.	OpenAI Moderation o Azure Content Safety.	Cuando los usuarios pueden subir imágenes y necesitas proteger la plataforma.
Multimodal	Combina imagen y texto para responder preguntas o razonar con contexto.	GPT-4o o Gemini.	

2.2 — Verdadero o Falso: APIs de Visión por Computador

Marca V (Verdadero) o F (Falso) y justifica brevemente las que sean Falsas:

V / F	Afirmación	Justificación si es Falsa
F	Las APIs modernas de visión requieren entrenar modelos propios con miles de imágenes.	Es falsa porque precisamente una de sus ventajas es que ya vienen preentrenadas y se usan directamente por API.
V	GPT-4o y Gemini pueden recibir texto e imagen a la vez en la misma petición.	
F	La visión clásica (OpenCV) es siempre mejor opción que las APIs modernas.	Es falsa porque depende del caso. Las APIs modernas son mejores para prototipos, tareas generales y multimodalidad; OpenCV o visión clásica conviene más si necesitas control total, trabajo offline o casos muy específicos.
V	La detección de objetos devuelve etiquetas, puntuación de confianza y coordenadas (bounding boxes).	
F	Hugging Face solo permite usar modelos de visión en la nube, no localmente.	Es falsa porque muchos modelos de Hugging Face pueden ejecutarse también en local mediante transformers o pipelines.
V	Para tareas de OCR simple, Gemini es una opción válida sin necesidad de entrenar nada.	

2.3 — ¿Qué técnica/API elegirías?

Para cada caso de uso, indica qué técnica CV y qué proveedor/API usarías, justificando brevemente:

Caso de uso	Técnica CV	Proveedor / API	¿Por qué?
Leer el importe y la fecha de un ticket de compra fotografiado	OCR	Gemini o Azure Vision	Porque la tarea consiste en extraer texto y datos concretos desde una imagen de documento.
Detectar si un usuario sube una imagen violenta en una red social	Moderación visual	OpenAI Moderation o Azure Content Safety	Porque necesitamos filtrar contenido sensible o peligroso antes de mostrarlo en la plataforma.
Una app que describa el entorno a personas con discapacidad visual	Descripción de imágenes	GPT-4o	Porque puede generar descripciones naturales, claras y útiles sobre lo que aparece en la escena.
Contar cuántos coches hay en un parking a partir de una foto aérea	Detección de objetos	YOLO o Hugging Face	Porque no solo hay que reconocer coches, sino localizarlos y contarlos dentro de la imagen.
Responder preguntas sobre el contenido de un cartel fotografiado	Multimodal	GPT-4o o Gemini	Porque la tarea requiere analizar la imagen y además entender la pregunta en texto para

			responder con contexto.
--	--	--	-------------------------

BLOQUE 3 — El Flujo de Visión por Computador en una App (3.2)

Vimos que el flujo CV tiene 5 fases. Define cada fase con tus palabras e indica su importancia:

Fase	Nombre	¿Qué se hace?	¿Por qué es necesaria?
1	Subida de imagen	El usuario sube una imagen, como una foto, captura, documento o producto.	Porque es el punto de entrada de la información visual que la app va a analizar.
2	Envío a la API	La aplicación envía la imagen a una API o modelo de visión por computador.	Porque el sistema necesita pasar la imagen al servicio que va a procesarla.
3	Análisis de la imagen	La API analiza la imagen y detecta objetos, texto, categorías o genera una descripción.	Porque aquí ocurre la interpretación real del contenido visual.
4	Respuesta estructurada	La API devuelve el resultado en un formato útil, normalmente texto o JSON.	Porque la app necesita una salida clara y procesable para poder actuar.
5	Acción en la aplicación	La aplicación usa el resultado para mostrar información, validar datos o activar una acción.	Porque el objetivo final no es solo analizar la imagen, sino usar ese análisis dentro del software.

3.2 — Investiga: Parámetros y decisiones clave al integrar CV

Investiga y define con tus propias palabras cada concepto. Incluye un ejemplo y cuándo aplicarlo:

Concepto	¿Qué controla/significa?	Valor / Ejemplo	¿Cuándo aplicarlo?
URL pública vs base64	Define cómo se envía la imagen al modelo: mediante un enlace accesible o codificada dentro de la petición.	URL pública: <code>https://.../imagen.jpg</code> ; base64: imagen convertida a texto codificado.	URL pública cuando la imagen ya está alojada online; base64 cuando la imagen es local, privada o no quieres depender de una URL externa.
Bounding box	Es el cuadro o coordenadas que indican dónde está un objeto dentro de la imagen.	<code>xmin=20, ymin=40, xmax=120, ymax=180</code>	Cuando necesitas localizar objetos, dibujar cajas o contar elementos en una imagen.
Score de confianza	Es la probabilidad o nivel de seguridad con el que el modelo cree que una detección o clasificación es correcta.	0.95 significa 95% de confianza.	Cuando quieres filtrar resultados poco fiables y quedarte solo con los que superan un umbral mínimo.
Prompt multimodal	Es la instrucción en texto que acompaña a una imagen para indicar qué debe hacer el modelo.	"Lee este cartel y explica qué significa".	Cuando no basta con analizar la imagen sola y necesitas guiar al modelo con una pregunta o tarea concreta.
Pipeline (Hugging Face)	Es una forma sencilla de cargar y usar un modelo preentrenado sin configurar todos los	<code>pipeline('object-detection')</code>	Cuando quieres probar rápidamente clasificación, OCR o detección de objetos

	detalles manualmente.		sin entrenar modelos desde cero.
--	--------------------------	--	-------------------------------------

BLOQUE 4 — Visión Clásica vs APIs Modernas (3.3)

Los embeddings en NLP tienen su equivalente en CV: modelos propios vs APIs. Responde:

Pregunta	Visión Clásica	APIs Modernas
¿Cuánto tarda en desarrollarse?	Suele tardar más, normalmente semanas o meses, porque hay que preparar datos, ajustar modelos y probar mucho.	Suele ser mucho más rápida, incluso en días, porque ya vienen listas para integrar.
¿Necesita dataset propio?	Sí, normalmente necesita imágenes propias para entrenar o ajustar el sistema.	No, en general no hace falta dataset propio porque los modelos ya están preentrenados.
¿Funciona sin conexión a internet?	Sí, puede funcionar en local y sin internet.	No, normalmente depende de una conexión para llamar a la API externa.
¿Cuál es su coste inicial?	Más alto, porque requiere tiempo, infraestructura, pruebas y a veces hardware específico.	Más bajo, porque se usa por pago por uso y no exige montar toda la infraestructura.
¿Requiere mantenimiento continuo?	Sí, porque hay que actualizar modelos, ajustar errores y mantener el pipeline.	Muy poco, porque el proveedor mantiene el modelo y la infraestructura.
¿Soporta razonamiento multimodal?	No de forma natural; habría que construir	Sí, muchas APIs modernas ya permiten

	mucho más sistema alrededor.	trabajar con imagen + texto en la misma petición
--	---------------------------------	--

4.2 — Investiga: Casos de uso reales de APIs de Visión

Busca información y describe un caso de uso real de visión por computador para cada sector:

Sector	Empresa / Producto real	¿Cómo usa la visión por computador?
Salud / Medicina	Google Cloud Healthcare / herramientas de análisis médico por imagen	Se usa para analizar radiografías, escáneres o imágenes clínicas y ayudar a detectar patrones anómalos.
Retail / E-commerce	Amazon Go	Utiliza visión por computador para identificar productos que el cliente toma y automatizar el proceso de compra sin cajas.
Seguridad vial / Transporte	Tesla Autopilot	Usa cámaras y visión por computador para detectar carriles, vehículos, señales y peatones en tiempo real.
Educación	Duolingo English Test	Utiliza visión por computador para supervisar visualmente al usuario durante el examen y detectar comportamientos sospechosos.

BLOQUE 5 — Pregunta de Síntesis e Investigación**Un compañero/a dice:**

"Para analizar imágenes en mi app, prefiero OpenCV: es gratis, funciona sin internet y no le paso mis datos a nadie. No necesito ninguna API externa."

¿Estás de acuerdo? Argumenta tu respuesta con al menos 3 razones concretas. Menciona al menos un caso donde la visión clásica SÍ sería la mejor opción:

Estoy parcialmente de acuerdo, pero no del todo. OpenCV y la visión clásica tienen ventajas reales, especialmente en privacidad, control y uso offline, pero no siempre son la mejor opción para una aplicación moderna.

En muchos proyectos reales, las APIs de visión actuales son más rápidas de integrar, más potentes en tareas generales y mucho más útiles si necesitamos razonamiento multimodal. Primera razón: las APIs modernas permiten desarrollar mucho más rápido. Con OpenCV o visión clásica muchas veces hay que construir bastante lógica manual, ajustar algoritmos y, en casos complejos, incluso entrenar modelos propios.

En cambio, una API moderna puede resolver en poco tiempo tareas como OCR, descripción de imágenes o preguntas sobre una foto. Segunda razón: las APIs modernas ya vienen preentrenadas y funcionan muy bien en tareas generales. Por ejemplo, leer tickets, describir escenas, moderar contenido o responder preguntas sobre una imagen. Hacer eso solo con OpenCV sería mucho más limitado, porque OpenCV por sí mismo no "entiende" imágenes como lo hace un modelo multimodal.

Tercera razón: las APIs modernas ofrecen capacidades multimodales, es decir, combinan imagen y texto en la misma petición. Esto permite casos como subir una foto de un cartel y preguntar "¿qué dice y qué significa?". Ese tipo de razonamiento contextual no es sencillo con visión clásica. Cuarta razón: aunque OpenCV sea gratis a nivel de licencia, el coste real no siempre es cero. Desarrollar, mantener, probar y mejorar un sistema clásico puede requerir mucho tiempo y esfuerzo técnico.

A veces una API de pago por uso sale más rentable si reduce semanas o meses de trabajo. Aun así, sí hay casos donde la visión clásica sería la mejor opción. Por ejemplo, en un sistema industrial que debe funcionar sin internet y detectar defectos muy concretos en piezas de fábrica con baja latencia y control total del proceso. También sería mejor opción en entornos con requisitos muy estrictos de privacidad, como sistemas militares, médicos internos o dispositivos embebidos sin conexión.

Conclusión: no elegiría una sola opción por principio. Para una app general, prototipos rápidos o tareas como OCR, descripción y multimodalidad, elegiría APIs modernas. Para sistemas offline, ultraspecíficos o con máxima privacidad, elegiría visión clásica. La mejor decisión depende del problema real que quiera resolver la aplicación.**



Visión por Computador — Fundamentos



Fundación Dicampus · Inclusión Socio-Laboral | Página 1